Inteligência Artificial 2025/26
Projeto: SLITHERLINK

23 de abril de 2026

1 Introdução

O projeto da unidade curricular de Inteligência Artificial (IArt) tem como objetivo desenvolver um programa em Python que resolva uma adaptação do problema SLITHERLINK[1] utilizando técnicas de procura e resolução de problemas de Inteligência Artificial.

SLITHERLINK, é chamado de Sli-Lin no Japão. Com regras simples baseadas apenas nos números 0, 1, 2 e 3, este puzzle desafia o jogador a criar um caminho contínuo que desliza pelo tabuleiro como um réptil, justificando o seu nome original Surizarinku (スリザーリンク). O desafio reside na descoberta de inúmeros teoremas lógicos que fazem com que o jogador se sinta cada vez mais dotado à medida que resolve novos desafios. Sendo o terceiro puzzle de lógica mais popular no Japão, a seguir ao Sudoku e ao Kakuro, o SLITHERLINK tornou-se um fenómeno cultural presente em revistas, jornais e até em consolas como a Nintendo DS.

O jogo pode ser encontrado em GooglePlay e AppStore.

2 Descrição do problema

O objetivo do SLITHERLINK é ligar pontos adjacentes numa grelha retangular para formar **um circuito único e contínuo**, sem cruzamentos ou ramificações. A linha do circuito deve respeitar as pistas numéricas no interior das células: cada número indica exatamente quantos dos seus quatro lados fazem parte do caminho. As células vazias podem ser rodeadas por qualquer número de linhas (máximo 3), desde que a solução final resulte num *loop* fechado e ininterrupto.

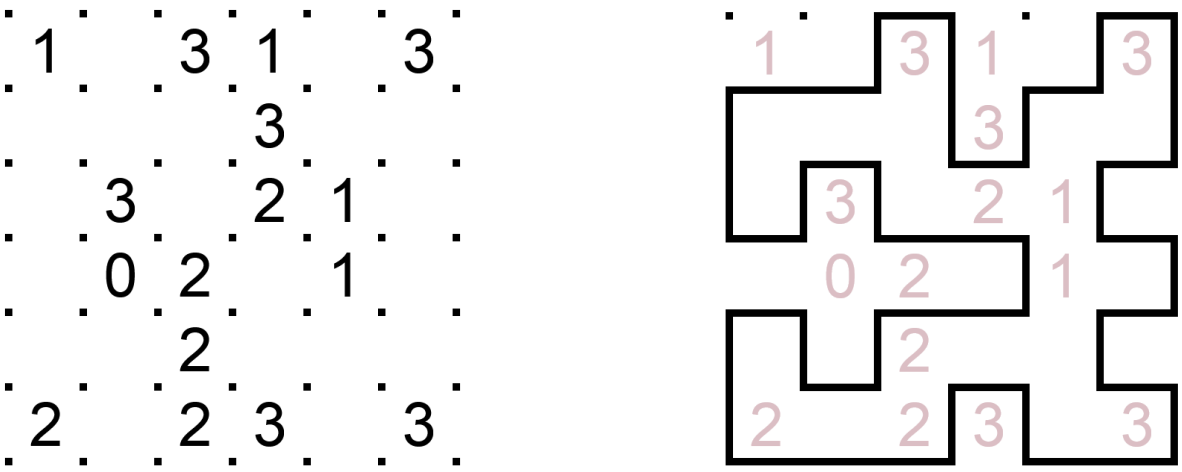
A Figura 1 baseia-se num puzzle disponível em Conceptis Puzzles. A Figura 1a mostra o estado inicial de um puzzle SLITHERLINK. Os pontos representam os vértices das células e os números no interior de cada célula indicam quantas das suas arestas fazem parte do circuito.

A Figura 1b consiste na solução no puzzle: um único circuito fechado, sem interseções, composto apenas por segmentos horizontais e verticais entre os pontos.

A Figura 2 apresenta possíveis passos iniciais na resolução do puzzle. Uma célula com valor zero indica que nenhuma das suas arestas faz parte do circuito. Para efeitos ilustrativos, o símbolo ‘x’ assinala arestas que não podem ser utilizadas. A rosa, indicam-se as células para

[1] https://en.wikipedia.org/wiki/Slitherlink

as quais já não existem arestas válidas a considerar. Os puzzles SLITHERLINK possuem uma solução única.



(a) Grelha inicial 6x6

(b) Solução

Figura 1: Exemplo do puzzle no estado inicial e depois de resolvido

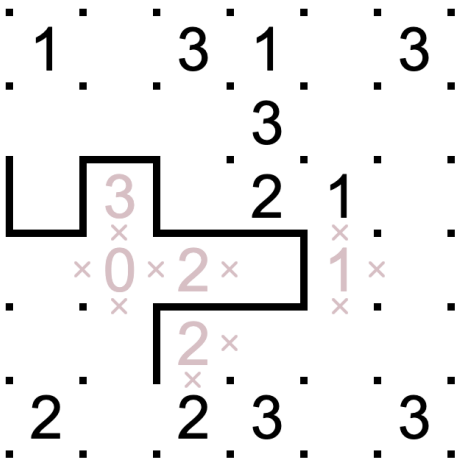


Figura 2: Passos intermédios na resolução do puzzle

3 Objetivo

O objetivo deste projeto é o desenvolvimento de um programa em Python (versão 3.11.4) que, dada uma instância de SLITHERLINK, retorna a solução (única), i.e., o ciclo fechado que

satisfaz todas as restrições numéricas da grelha e as propriedades topológicas do puzzle.

O programa deve ser desenvolvido num ficheiro `slitherlink.py`, que lê uma instância de SLITHERLINK a partir do *standard input* no formato descrito na secção 4.1. O programa deve resolver o problema utilizando uma técnica à escolha e imprimir a solução para o *standard output* no formato descrito na secção 4.2.

4 Formato de input e output

Uma grelha de SLITHERLINK tem dimensões N x M. As células da grelha podem ou não ter números que descrevem as restrições. O programa a desenvolver deve ler um ficheiro de texto no formato de input e devolver o formato de output.

4.1 Formato do input

As instâncias do problema SLITHERLINK seguem o seguinte formato:

```
<cell-value-l1c1> ... <cell-value-l1cN>
...
<cell-value-lNc1> ... <cell-value-lNcN>
```

Os valores possíveis para `<cell-value-*>` são os números 0, 1, 2 ou 3 indicando o número de restrições da célula. Células que não têm uma restrição explícita, e podem ou não fazer parte do circuito, são representadas com '.'.

4.1.1 Exemplo

O ficheiro de input que descreve a instância da Figura 1a é o seguinte:

```
1 . 3 1 . 3
. . . 3 . .
. 3 . 2 1 .
. 0 2 . 1 .
. . 2 . . .
2 . 2 3 . 3
```

```
1\t.\t3\t1\t.\t3\n
.\t.\t.\t3\t.\t.\n
.\t3\t.\t2\t1\t.\n
.\t0\t2\t.\t1\t.\n
.\t.\t2\t.\t.\t.\n
2\t.\t2\t3\t.\t3\n
```

A leitura através do *standard input* faz-se da seguinte forma:

```
python slitherlink.py < testN.txt
```

4.2 Formato do output

O output do programa deve representar uma solução para o problema de SLITHERLINK. Para cada célula da grelha, devem ser indicadas quais das suas arestas fazem parte da solução (arestas ativas). O output deve seguir o seguinte formato:

- conter uma linha por cada linha da grelha.
- cada linha contém a representação de todas as células dessa linha.

Cada célula é representada por uma sequência de 4 dígitos binários (0 ou 1): *top, right, bottom, left*. 1 indica que a aresta da *cell-lNcN* faz parte da solução (está ativa), 0 significa que a aresta está inativa.

4.2.1 Exemplo

O output que descreve a solução da Figura 1b é:

```
0010 0010 1101 0001 0010 1101
1001 1010 0100 0111 1001 0110
0111 1101 0011 1010 0100 1011
1010 0000 1010 1110 0001 1110
1101 0111 1001 1010 0100 1011
0011 1010 0110 1101 0011 1110
```

```
0010\t0010\t1101\t0001\t0010\t1101\n
1001\t1010\t0100\t0111\t1001\t0110\n
0111\t1101\t0011\t1010\t0100\t1011\n
1010\t0000\t1010\t1110\t0001\t1110\n
1101\t0111\t1001\t1010\t0100\t1011\n
0011\t1010\t0110\t1101\t0011\t1110\n
```

5 Implementação

Para a realização deste projeto devem ser utilizados os ficheiros Python, disponibilizados na página da unidade curricular, que implementam os algoritmos de procura que irão ser dados

ao longo da época letiva [2]. O mais importante é compreender para que servem e como usar as funcionalidades implementadas nestes ficheiros.

Os ficheiros search.py e utils.py não devem ser alterados. Se houver necessidade de alterar definições incluídas nestes ficheiros, estas alterações devem ser feitas no ficheiro de código desenvolvido pelo vosso grupo que contém a implementação do projeto. A solução implementada deve usar as funcionalidades descritas no ficheiro slitherlink.py.

Outras dependências não são permitidas, excepto o Python package `numpy`, que pode ser útil para representar a solução e ter acesso a operações sobre arrays.

5.0.1 Procuras

No ficheiro `search.py` estão implementadas as estruturas necessárias para correr os diferentes algoritmos de procura. Destacam-se:

- Classe `Problem`: Representação abstrata do problema de procura;
- Função `breadth_first_tree_search`: Procura em largura primeiro;
- Função `depth_first_tree_search`: Procura em profundidade primeiro;
- Função `greedy_search`: Procura gananciosa;
- Função `astar_search`: Procura A*.

5.0.2 Classe `SlytherlinkState`

A classe *SlytherlinkState* define o objeto que representa o estado do tabuleiro em cada instante. O membro `board` armazena a configuração da grelha a que o estado corresponde.

Abaixo é apresentado o código desta classe. Podem ser feitas alterações a esta classe, como por exemplo modificações ao método `__lt__(self, other)` para suportar funções de desempate mais complexas. No entanto, estas alterações devem ser devidamente justificadas com comentários no código.

```
class SlytherlinkState:
    state_id = 0

    def __init__(self, board):
        self.board = board
        self.id = Slitherlink.state_id
        Slitherlink.state_id += 1

    def __lt__(self, other):
        """ Este método é utilizado em caso de empate na gestão da lista
```

[2] Este código é adaptado a partir do código disponibilizado com o livro *Artificial Intelligence: a Modern Approach* e que está disponível em https://github.com/aimacode.

```
        de abertos nas procuras informadas. """
        return self.id < other.id
```

5.1 Código a implementar

5.1.1 Classe `Board`

A classe `Board` é a representação interna de uma grelha de SLITHERLINK. A implementação desta classe e respectivos métodos é livre.

Pode, a título de exemplo, incluir os métodos para determinar células ou valores adjacentes `adjacent_cell` e `get_cell_edges`. A primeira função recebe o tuplo com as coordenadas de uma célula e devolve uma lista com as coordenadas das células adjacentes. A segunda função também recebe um tuplo com as coordenadas de uma célula e devolve uma lista das arestas dessa célula.

Pode também implementar outros métodos, como por exemplo um método `get_active_edges` que retorne o valor de arestas ativas numa determinada célula, ou um método `print_instance` que imprime a grelha no formato descrito na secção 4.2.

Estes métodos poderão ser utilizados para fazer testes à restante implementação da classe.

```
class Board:
    """Representação interna de um tabuleiro do Slytherlink."""

    def adjacent_cell(self, cell:tuple) -> list:
        """Devolve uma lista das células que fazem
        fronteira com a célula enviada no argumento"""
        #TODO
        pass

    def get_cell_edges(self, row:int, column:int) -> list:
        """Devolve as arestas da célula enviada no argumento"""
        #TODO
        pass

    def get_active_edges(self, row:int, column:int) -> list:
        """Devolve o número de arestas ativas"""
        #TODO
        pass

    # TODO: outros metodos da classe
```

5.1.2 Função `parse_instance`

A função `parse_instance` é responsável por ler uma instância do problema no formato de input apresentado (secção 4.1) e devolver um objeto do tipo `Board` que a represente. Esta função deve ler a instância a partir do *standard input* (stdin).

```
@staticmethod
def parse_instance():
    """Lê a instância do problema do standard input (stdin)
    e retorna uma instância da classe Board.

    Por exemplo:
        $ python3 pipe.py < test-01.txt

        > from sys import stdin
        > line = smtdin.readline().split()
    """
    # TODO
    pass
```

5.1.3 Classe `Slytherlink`

A classe `Slytherlink` herda da classe `Problem` definida no ficheiro `search.py` do código a utilizar e deve implementar os métodos necessários ao seu funcionamento.

O método `actions` recebe como argumento um estado e retorna uma lista de ações que podem ser executadas a partir desse estado. O método `result` recebe como argumento um estado e uma ação, e retorna o resultado de aplicar essa ação a esse estado.

Pode, a título de exemplo, considerar que uma ação corresponde a marcar uma aresta como ativa.

Para suportar as procuras informadas, nomeadamente a procura gananciosa e a procura A*, deve desenvolver uma heurística que consiga guiar da forma mais eficiente possível estas procuras. A heurística corresponde à implementação do método `h` da classe `Slytherlink`. Esta função recebe como argumento um `node`, a partir do qual se pode aceder ao estado atual em `node.state`.

De seguida é disponibilizado um protótipo da classe `Slytherlink` que pode ser usado como base para a sua implementação.

```
class Slytherlink(Problem):
    def __init__(self, initial_state: Board):
        """ O construtor especifica o estado inicial. """
        # TODO
        pass
```

```python
    def actions(self, state: State):
        """ Retorna uma lista de ações que podem ser executadas a
        partir do estado passado como argumento. """
        # TODO
        pass

    def result(self, state: State, action):
        """ Retorna o estado resultante de executar a 'action' sobre
        'state' passado como argumento. A ação a executar deve ser uma
        das presentes na lista obtida pela execução de
        self.actions(state). """
        # TODO
        pass

    def h(self, node: Node):
        """ Função heuristica utilizada para a procura A*. """
        # TODO
        pass
```

5.1.4 Exemplos de utilização

De seguida, são apresentados alguns exemplos da utilização do código a desenvolver, assim como o respetivo output. Estes exemplos podem ser utilizados para testar a implementação. Considere que o ficheiro `test02.txt` se encontra na diretoria "./slitherlink-boards-public/"e que contém a instância descrita na secção 4.1.

Exemplo 1:

```python
# Ler grelha da figura 1a:
board = Board.parse_instance()

print(board.get_cell_edges(3,1))
print(board.get_cell_edges(4,4))
```

Output:

```
[('h', 3, 1), ('h', 4, 1), ('v', 3, 1), ('v', 3, 2)]
[('h', 4, 4), ('h', 5, 4), ('v', 4, 4), ('v', 4, 5)]
```

Exemplo 2:

```
# Ler grelha da figura 1a:
board = Board.parse_instance()

# Criar uma instância de Slytherlink:
problem = Slytherlink(board)

# Criar um estado com a configuração inicial:
initial_state = SlytherlinkState(board)

print(result_state.board.get_inactive_edges(2, 1))

# Realizar ação de ativar as arestas da célula (2, 1)
result_state = s1 = problem.result(initial_state,[('h', 2, 1),
                                                  ('v', 2, 1),
                                                  ('v', 2, 2)])

# Mostrar valor na posição (2, 1):
print(result_state.board.get_active_edges(2, 1))
print(result_state.board.get_inactive_edges(2, 1))
```

Output:

```
1
3
1
```

Exemplo 3:

```
# Ler grelha da figura 1a:
board = Board.parse_instance()

# Criar uma instância de Slytherlink:
problem = Slytherlink(board)

# Criar um estado com a configuração inicial:
s0 = SlytherlinkState(board)

# Aplicar as ações que resolvem a instância
s1 = problem.result(s0, [('h', 2, 1), ('v', 2, 1), ('v', 2, 2)])
s2 = problem.result(s1, [('h', 3, 0)]
s3 = problem.result(s2, [('h', 3, 2)]

# Verificar se foi atingida a solução
print("Is goal?", problem.goal_test(s2))
print("Is goal?", problem.goal_test(s3))
print("Actual State:\n", s3.board.print(), sep="")
```

Output:

```
Is goal? False
Is goal? False
Actual State:
0000 0000 0000 0000 0000 0000
0000 0000 0000 0000 0000 0000
0010 1101 0011 0000 0000 0000
0000 0000 0000 0000 0000 0000
0000 0000 0000 0000 0000 0000
0000 0000 0000 0000 0000 0000
```

Exemplo 4:

```
# Ler grelha da figura 1a:
board = Board.parse_instance()

# Criar uma instância de Slytherlink:
problem = Slytherlink(board)

# Obter o nó solução usando a procura em profundidade:
goal_node = depth_first_tree_search(problem)

# Verificar se foi atingida a solução
print("Is goal?", problem.goal_test(goal_node.state))
print("Solution:\n", goal_node.state.board.print(), sep="")
```

Output:

```
Is goal? True
Solution:
0010 0010 1101 0001 0010 1101
1001 1010 0100 0111 1001 0110
0111 1101 0011 1010 0100 1011
1010 0000 1010 1110 0001 1110
1101 0111 1001 1010 0100 1011
0011 1010 0110 1101 0011 1110
```

O valor de retorno das funções de procura é um objeto do tipo `Node`. Do nó de retorno podem ser retiradas as diversas informações, por exemplo, estado final (`goal_node.state`), a ação que levou ao estado final `goal_node.action`, e o nó precedente `goal_node.parent`.

5.1.5 Ficheiro `slitherlink_gui.py`

O ficheiro `slitherlink_gui.py` descreve a `Class SlitherlinkGUI`, implementa uma representação visual do SLITHERLINK jogável.

É possivel usar a visualização para suporte ao desenvolvimento e debug. No ficheiro `slitherlink.py` devem criar o objeto SlitherlinkGUI. Exemplo de utilização:

```
# Criar a GUI antes de iniciar o vosso program
root = tk.Tk()
app  = SlitherlinkGUI(root, game_board.board)

# lógica de resolução

app.load_solution(sol_grid)
```

Para visualização dinâmica cada vez que uma ação é executa é necessário alterar o construtor da classe `Slytherlink`.

```
class Slytherlink(Problem):
def __init__(self, initial_state: Board, gui=None):
    """ O construtor especifica o estado inicial. """
    # TODO
    self.gui = gui   # optional SlitherlinkGUI instance for live visualisation
```

6 Avaliação

A nota do projecto será baseada nos seguintes critérios:

- Execução correcta (70% - 14 val.). Estes valores correspondem a testes realizados via submissão no GitLab.
- Relatório (30% - 6 val.). O relatório será realizado em formato vídeo.

7 Condições de realização e prazos

- O projecto deve ser realizado em grupos de 2 alunos.
- Publicação do enunciado: até ao dia 24 de Abril
- Inscrições de grupos no Fénix: até às 17:00 de dia 30 de Abril
- Entrega do projeto (.py) no GitLab interno: 29 de Maio, até às 17:00
- Entrega do relatório no Microsoft Teams: 29 de Maio, até às 23:59

As inscrições dos grupos para o projeto serão feitas através do Fénix; este passo é essencial para posterior acesso ao Gitlab da RNL. O código do projeto e relatório têm de ser entregues obrigatoriamente por via electrónica. O código do projeto será entregue através da plataforma *gitlab.rnl.tecnico.ulisboa.pt* e o relatório através do Microsoft Teams.

7.1 Planeamento

Sugere-se que cada grupo planeie atempadamente a realização do projeto, por exemplo tendo em conta as seguintes metas:

- **Até ao dia 8 de Maio**: Leitura de input / instância de exemplo.
- **Até ao dia 15 de Maio**: Modelação do problema e definição do que é uma ação no contexto do projeto.
- **Até ao dia 22 de Maio**: Implementação das funções de geração de ações e resultados de aplicar uma ação a um estado.

- **Até ao dia 26 de Maio**: Experimentar diferentes procuras e obter soluções corretas para instâncias iniciais (validar formato de output).

7.2 GitLab.rnl

A avaliação da execução do código do projecto será feita automaticamente através da plataforma GitLab.rnl Após o prazo de inscrição no Fénix e quando notificado pelo corpo docente siga as seguintes instruções para registar e submeter no Gitlab:

- **A partir do dia 12 de Maio**, será possível acederem ao Gitlab da RNL para submissão do projeto.
- Para conseguirem aceder a *gitlab.rnl.tecnico.ulisboa.pt* devem garantir em *ciist.ist.utl.pt* que o serviço *afs* está *enabled*. As credenciais de acesso são as mesmas do fénix.
- Deverá ser submetido o ficheiro *slitherlink.py* contendo o código do seu projecto. O ficheiro de código deve conter em comentário, nas primeiras linhas, o número e o nome dos alunos. **É obrigatório**.
- Quando a submissão tiver sido processada, poderá visualizar o resultado correspondente.

Submeta o seu projeto atempadamente, dado que as restrições seguintes podem não lhe permitir fazê-lo no último momento:

- Até ao prazo de entrega poderá efectuar o número de entregas que desejar, sendo utilizada para efeitos de avaliação a **última entrega efectuada**.
- Os testes considerados para efeitos de avaliação podem incluir ou não os exemplos disponibilizados, além de um conjunto de testes adicionais.
- O tempo de execução de cada teste está limitado, bem como a memória utilizada.

7.3 Relatório em formato de vídeo

Deve ser produzido um relatório em formato de vídeo com **duração máxima de 4 minutos**. Todos os elementos do grupo devem participar ativamente no vídeo, explicando partes específicas do trabalho. A exposição deve evidenciar domínio do raciocínio subjacente às decisões tomadas.

O vídeo deve ser estruturado, claro e percetível, fazendo uso de material de apoio ilustrativo que não necessita de ser sofisticado (por exemplo: slides, imagens, esquemas ou pequenas demonstrações). No final, é obrigatória a inclusão de um slide de disclaimer que descreva de forma transparente como foram utilizadas ferramentas de Inteligência Artificial Generativa.

Seguem-se algumas sugestões para o conteúdo do vídeo, que podem ser utilizadas se consideradas adequadas:

- Breve descrição do problema (regras e objetivo).
- Descrição da ideia geral para abordar o problema.

- Identificação do que constitui uma ação no contexto do problema.
- Descrição do resultado da aplicação de uma ação a um estado.
- Pré-processamento ou métodos de inferência utilizados.
- Pequena avaliação experimental comparando diferentes estratégias de procura nas instâncias disponibilizadas.
- Identificação da abordagem final selecionada, incluindo as suas características (por exemplo, heurísticas) e a justificação da escolha.
- Discussão das alternativas exploradas e das decisões tomadas, nomeadamente ao nível de estruturas de dados e algoritmos utilizados (incluindo métricas como o número de nós gerados e expandidos).
- Heurísticas desenvolvidas, o respetivo racional e o impacto no desempenho.

Espera-se ainda que o vídeo integre exemplos concretos do processo de desenvolvimento, incluindo iterações, testes realizados, erros encontrados e respetivas correções. Todo o conteúdo deve refletir explicitamente, e de forma crítica, os conceitos abordados ao longo da unidade curricular no contexto da solução implementada. A avaliação privilegiará a qualidade da análise crítica e a justificação das decisões, em detrimento da mera descrição de soluções.

O vídeo deve ser apresentado a velocidade normal, de forma a assegurar a clareza da exposição. A utilização de aceleração para cumprir o limite de tempo não é permitida, sendo o seu incumprimento sujeito a penalização.

Grupos que não disponham dos meios básicos necessários para a realização do vídeo (i.e., computador com câmara, microfone e acesso a ferramentas de gravação como o Zoom) deverão contactar o corpo docente.

8 Discussões

O corpo docente reserva-se o direito de convocar qualquer grupo para uma discussão oral sobre o trabalho desenvolvido, se considerar necessário, para efeitos de esclarecimento e validação da autoria.

9 Cópias

Projectos iguais, ou muito semelhantes, originarão a reprovação na disciplina e, eventualmente, o levantamento de um processo disciplinar. Para o efeito será usada a ferramenta Moss da Universidade de Stanford [3].

Os programas entregues serão também testados em relação a soluções existentes na web.

[3] https://theory.stanford.edu/~aiken/moss/

As analogias encontradas com os programas da web, incluindo soluções produzidas por *Generative AI*, serão tratadas como cópias.

10 Uso de IA Generativa

Reconhece-se que ferramentas de IA generativa (como ChatGPT, Gemini, Cursor, entre outras) são amplamente utilizadas e podem apoiar o processo de aprendizagem. No entanto, o trabalho submetido deve ser original e da autoria dos estudantes. Neste contexto, a IA deve ser encarada como uma ferramenta de apoio ou consulta (semelhante a um colega), e não como autora do trabalho.

Neste projeto aplica-se o **Uso Limitado com Declaração de IA Generativa – Nível 1 (Amarelo)**, segundo as regras do "Guia para a Utilização Responsável da Inteligência Artificial", publicado pelo Instituto Superior Técnico, que recomendamos a leitura.

No slide de *disclaimer* (ver Secção 7.3), explicar de forma clara como as ferramentas foram utilizadas e de que modo os seus contributos foram avaliados e integrados no trabalho final. A submissão desta reflexão **é obrigatória**, independentemente de ter havido utilização de IA ou não.

Exemplo 1 Reconheço a utilização de [ferramenta de IA + versão, entidade responsável e URL] para gerar [resumo do que foi gerado].

Exemplo 2 Reconheço a utilização de [ferramenta de IA + versão, entidade responsável e URL] para resumir [fonte x], que depois [descrever a ação realizada].

Exemplo 3 Introduzi o seguinte prompt [escrever o prompt entre aspas ou em itálico] em [ferramenta de IA + versão, entidade responsável e URL], que produziu o resultado X. Em seguida, [descrever como utilizou qualquer parte do resultado no seu trabalho].

Recorda-se que o objetivo do projeto é treinar a mente para pensar.